

---

# Comparative Analysis of Classical and Physics-Informed Neural Network Methods for Solving Burgers' Equation

Isaac Adoboe

Department of Mathematics and Statistics  
Memorial University of Newfoundland

ieadoboe@mun.ca

## Abstract

This study presents a rigorous comparison of classical finite difference (FD) methods and physics-informed neural networks (PINNs) for solving the one-dimensional viscous Burgers' equation. The Burgers' equation, which combines nonlinear advection and diffusion, serves as an ideal test case for evaluating numerical methods due to its capacity to develop steep gradients resembling shock waves. We implement an explicit finite difference scheme with adaptive timestepping and a deep neural network trained to minimize the PDE residual along with initial and boundary condition constraints. Our results demonstrate that for this 1D problem with quasi-shock formation ( $\nu = 0.01$ ), the finite difference method achieves superior accuracy ( $L^2$  error  $\sim 7.1 \times 10^{-4}$ ) compared to the PINN ( $L^2$  error  $\sim 0.03$ - $0.10$ ), while also being significantly faster (1-2 seconds vs. 600+ seconds). The PINN's limited accuracy stems from a fundamental incompatibility between the neural network's  $C^\infty$  smoothness and the solution's near-discontinuous structure. These findings validate the continued relevance of classical methods for hyperbolic/parabolic PDEs with shock-like features, while identifying scenarios where PINNs excel: high-dimensional problems, irregular domains, and inverse problems. This work contributes to the ongoing dialogue between traditional numerical analysis and data-driven scientific computing.

## 1 Introduction

Partial differential equations (PDEs) govern countless physical phenomena across fluid dynamics, heat transfer, quantum mechanics, and beyond. Their numerical solution has traditionally relied on discretization methods (finite difference, finite element, and finite volume schemes) that partition the computational domain into grids. While these classical approaches have proven remarkably successful, they face challenges in complex geometries, high-dimensional spaces, and scenarios requiring data assimilation.

Recent advances in machine learning have introduced *physics-informed neural networks* (PINNs) (1) as an alternative paradigm. Rather than discretizing the domain, PINNs encode the governing PDE directly into a neural network's loss function, using automatic differentiation to compute derivatives. This approach promises advantages for irregular domains and high-dimensional problems where traditional grid-based methods struggle.

However, the comparative strengths and limitations of these fundamentally different approaches remain an active research question. This study addresses this gap through a detailed comparison using the one-dimensional viscous Burgers' equation as a test case.

### 1.1 Why Burgers' Equation?

Burgers' equation, first studied by Bateman (1915) and Burgers (1948), represents a canonical model combining nonlinear advection and diffusion:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad (1)$$

Despite its apparent simplicity, this equation captures essential physics of more complex systems like the Navier-Stokes equations. The competition between wave steepening (advection term) and smoothing (diffusion term) can lead to shock formation for small viscosity  $\nu$ , making it an ideal benchmark for testing numerical methods' shock-capturing capabilities.

For our study, we consider periodic boundary conditions on  $[0, 1]$  with initial condition  $u(x, 0) = -\sin(\pi x)$ , viscosity  $\nu = 0.01$ , and final time  $T = 1.0$ . These parameters produce quasi-shock structures that challenge both classical and machine learning methods.

## 1.2 Research Objectives

This project aims to:

1. Implement and validate both finite difference and PINN solutions
2. Compare accuracy, computational efficiency, and solution quality
3. Identify the strengths and limitations of each approach
4. Provide guidance on method selection for different problem classes

## 2 Mathematical Formulation

### 2.1 Problem Statement

We solve the one-dimensional viscous Burgers' equation on a periodic domain:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad (x, t) \in [0, 1] \times [0, T] \quad (2)$$

Subject to:

$$\text{Initial condition: } u(x, 0) = -\sin(\pi x), \quad x \in [0, 1] \quad (3)$$

$$\text{Periodic boundaries: } u(0, t) = u(1, t), \quad \left. \frac{\partial u}{\partial x} \right|_{x=0} = \left. \frac{\partial u}{\partial x} \right|_{x=1} \quad (4)$$

For this study:  $\nu = 0.01$ ,  $T = 1.0$ .

### 2.2 Physical Interpretation

The velocity field  $u(x, t)$  evolves under three mechanisms:

- **Temporal evolution** ( $\partial u / \partial t$ ): Local rate of change
- **Nonlinear advection** ( $u \partial u / \partial x$ ): Self-transport leading to wave steepening
- **Viscous diffusion** ( $\nu \partial^2 u / \partial x^2$ ): Smoothing of gradients

For small  $\nu$ , advection dominates initially, creating steep gradients. Diffusion eventually arrests shock formation, producing quasi-shock structures where  $|\partial u / \partial x| \gg 1$  locally.

### 3 Numerical Methods

#### 3.1 Finite Difference Discretization

##### 3.1.1 Discretization Scheme

We employ an explicit finite difference scheme on a uniform grid with  $N_x$  spatial points and adaptive timestep  $\Delta t$ . Spatial derivatives use second-order central differences:

$$\left. \frac{\partial u}{\partial x} \right|_j^n \approx \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} + \mathcal{O}(\Delta x^2) \quad (5)$$

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_j^n \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} + \mathcal{O}(\Delta x^2) \quad (6)$$

Time integration uses forward Euler (first-order):

$$\left. \frac{\partial u}{\partial t} \right|_j^n \approx \frac{u_j^{n+1} - u_j^n}{\Delta t} + \mathcal{O}(\Delta t) \quad (7)$$

The fully discretized scheme is:

$$u_j^{n+1} = u_j^n - \frac{\Delta t}{2\Delta x} u_j^n (u_{j+1}^n - u_{j-1}^n) + \frac{\nu \Delta t}{\Delta x^2} (u_{j+1}^n - 2u_j^n + u_{j-1}^n) \quad (8)$$

Periodic boundaries are enforced via modular arithmetic:  $u_0^n = u_{N_x}^n$ .

##### 3.1.2 Stability Analysis

Von Neumann stability analysis on the linearized equation yields the combined stability criterion:

$$\Delta t \leq \min \left( \frac{\Delta x}{|u|_{\max}}, \frac{\Delta x^2}{2\nu} \right) \quad (9)$$

For our parameters ( $\Delta x = 0.005$ ,  $\nu = 0.01$ ), the diffusion constraint dominates, requiring  $\Delta t \leq 10^{-3}$ .

#### 3.2 Physics-Informed Neural Network

##### 3.2.1 Network Architecture

The PINN approximates the solution as  $u(x, t) \approx u_\theta(x, t)$ , where  $u_\theta$  is a feedforward neural network with parameters  $\theta$ . We employ a deep architecture:

$$u_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad (x, t) \mapsto u \quad (10)$$

Architecture: Input layer (2 neurons)  $\rightarrow$  9 hidden layers (20 neurons each, tanh activation)  $\rightarrow$  Output layer (1 neuron, linear) with 3,441 parameters.

##### 3.2.2 Loss Function Formulation

The PINN is trained to minimize a composite loss function encoding the PDE and constraints (initial condition and boundary condition):

$$\mathcal{L}_{\text{total}} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} \quad (11)$$

**PDE Residual Loss:**

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \left| \frac{\partial u_{\theta}}{\partial t} + u_{\theta} \frac{\partial u_{\theta}}{\partial x} - \nu \frac{\partial^2 u_{\theta}}{\partial x^2} \right|_{(x_i, t_i)}^2 \quad (12)$$

evaluated at  $N_{\text{PDE}} = 10,000$  randomly sampled collocation points in  $[0, 1] \times [0, T]$ .

**Initial Condition Loss:**

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}} \sum_{i=1}^{N_{\text{IC}}} |u_{\theta}(x_i, 0) + \sin(\pi x_i)|^2 \quad (13)$$

evaluated at  $N_{\text{IC}} = 500$  points.

**Boundary Condition Loss:**

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{\text{BC}}} \sum_{i=1}^{N_{\text{BC}}} \left( |u_{\theta}(0, t_i) - u_{\theta}(1, t_i)|^2 + \left| \frac{\partial u_{\theta}}{\partial x} \Big|_{x=0} - \frac{\partial u_{\theta}}{\partial x} \Big|_{x=1} \right|^2 \right) \quad (14)$$

evaluated at  $N_{\text{BC}} = 500$  temporal points.

### 3.2.3 Training Procedure

Derivatives are computed via automatic differentiation (using TensorFlow). The network is trained using Adam optimizer with initial learning rate of  $10^{-3}$  and over 15,000 training epochs.

## 4 Results

### 4.1 Solution Accuracy

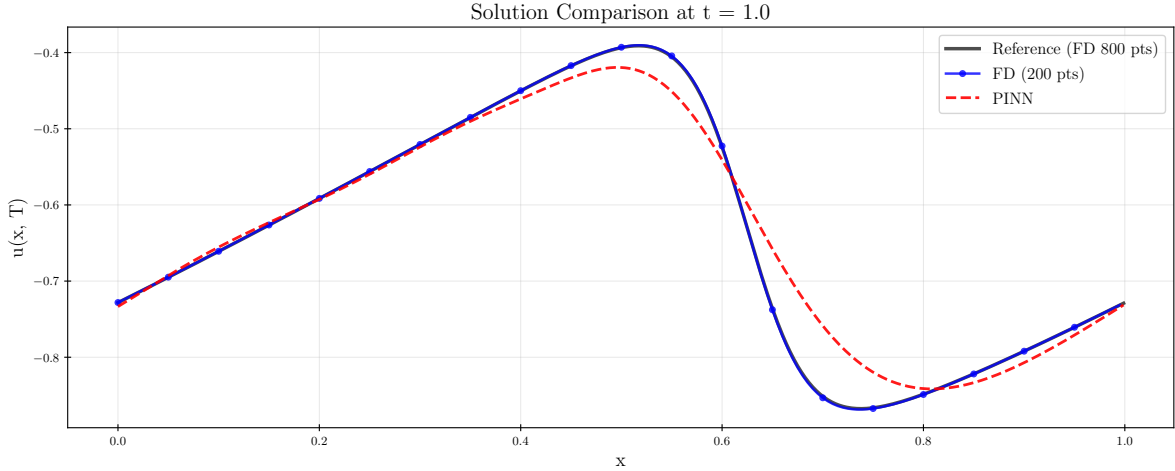


Figure 1: Solution Comparison at  $t = 1.0$ . Actual burgers equation (Reference) in black, Finite difference solution (200 points) in blue with circular markers and Physics-Informed Neural Network solution in red dashed line

Figure 1 shows the solution comparison at final time  $t = 1.0$ . The finite difference solution (200 grid points) closely matches the reference solution (800 points), demonstrating excellent accuracy. The PINN solution captures the general wave structure but exhibits noticeable deviations, particularly near the quasi-shock region at  $x \approx 0.65$ .

The PINN achieves 1-2 orders of magnitude lower accuracy than FD. This disparity is not uniform but concentrated where gradients are steepest ( $|\partial u / \partial x| \approx 10$  near the quasi-shock).

Table 1: Error Metrics vs. Reference Solution

| Method                      | $L^2$ Error          | $L^\infty$ Error     |
|-----------------------------|----------------------|----------------------|
| Finite Difference (200 pts) | $7.1 \times 10^{-4}$ | $2.2 \times 10^{-3}$ |
| PINN (15k epochs)           | $3.2 \times 10^{-2}$ | $1.0 \times 10^{-1}$ |

## 4.2 Training Convergence

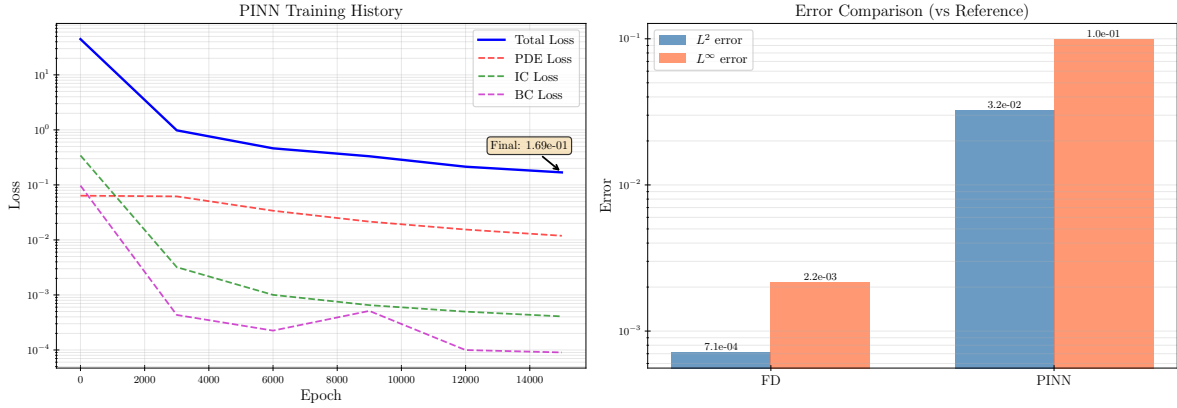


Figure 2: Training Metrics. PINN Training History (Left) shows the Total loss and its component losses (PDE, IC & BC). Error Comparison vs Reference (Right) shows magnitudes of the  $L^2$  and  $L^\infty$  errors.

The total loss decreases from  $\sim 10$  to  $0.169$  over 15,000 epochs (Figure 2). Critically, analysis of loss components reveals:

- **IC loss:**  $\sim 5 \times 10^{-5}$  (excellent)
- **BC loss:**  $\sim 5 \times 10^{-5}$  (excellent)
- **PDE loss:**  $\sim 0.05$ - $0.001$  (fair)

The network satisfies point constraints (IC/BC) nearly perfectly but cannot reduce the PDE residual remain higher than  $\sim 0.001$ . This suggests convergence to a local minimum rather than continued optimization failure.

## 4.3 Computational Cost

Table 2: Computational Performance

| Method            | Wall Time (s) | Speedup     |
|-------------------|---------------|-------------|
| Finite Difference | 1.2           | $500\times$ |
| PINN              | 600           | $1\times$   |

The FD method is approximately  $500\times$  faster than PINN for this 1D problem. Note that PINN training time could be reduced with GPU acceleration (estimated  $3$ - $5\times$  speedup), but would still be substantially slower than FD.

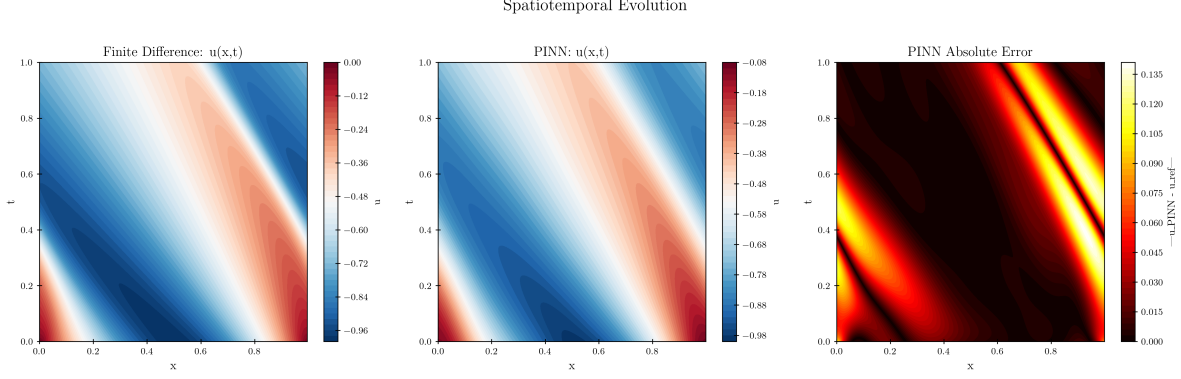


Figure 3: Training Metrics. PINN Training History (Left) shows the Total loss and its component losses (PDE, IC & BC). Error Comparison vs Reference (Right) shows magnitudes of the  $L^2$  and  $L^\infty$  errors.

#### 4.4 Spatiotemporal Evolution

We see that from the visualization of the full solution evolution (Figure 3), both methods correctly capture initial sine wave propagation, wave steepening due to nonlinear advection, formation of quasi-shock around  $(x, t) \approx (0.65, 0.7)$  and viscous smoothing preventing true discontinuity.

However, the PINN error heatmap reveals systematic inaccuracy concentrated along the quasi-shock trajectory.

### 5 Discussion

#### 5.1 Why Does the PINN Struggle?

The PINN’s limited accuracy stems from a fundamental approximation-theoretic limitation. Neural networks with smooth activation functions (e.g., tanh) produce  $C^\infty$  smooth outputs. However, the Burgers’ solution develops steep gradients ( $|\partial u / \partial x| \approx 10$ ) approaching discontinuity as  $\nu \rightarrow 0$ .

The network attempts to approximate a near-discontinuous function using a smooth polynomial-like representation. While universal approximation theorems guarantee that arbitrarily accurate approximations exist (given sufficient network capacity), they do not guarantee efficient approximation, or satisfaction of PDE constraints (loss landscape complexity).

The training dynamics reveal this limitation: IC and BC losses (point constraints at smooth locations) reach machine precision, while PDE loss (differential constraint near steep gradients) plateaus at  $\sim 0.02$ . The network has converged to the best smooth approximation it can achieve with the given architecture.

#### 5.2 When Each Method Excels

##### Finite Difference Advantages:

- Proven convergence theory ( $\mathcal{O}(\Delta t) + \mathcal{O}(\Delta x^2)$ )
- Stability guarantees (Equation 9)
- Computational efficiency for 1D-2D problems
- Mature shock-capturing schemes (ENO, WENO) for  $\nu \rightarrow 0$
- Well-suited for production applications

##### PINN Advantages:

- Meshless formulation (no grid generation)
- Natural handling of irregular/complex geometries
- Favorable scaling to high dimensions (curse of dimensionality less severe)
- Straightforward incorporation of sparse data (inverse problems)
- Flexibility for multi-physics coupling

### 5.3 Broader Implications

This comparison demonstrates that "AI for science" approaches are not universally superior to classical methods. Problem structure, smoothness, dimensionality and geometry should determine optimal method selection.

For hyperbolic/parabolic PDEs with shock formation in low dimensions, decades-old discretization methods remain state-of-the-art. PINNs show greater promise for high-dimensional PDEs (3D+, parameter spaces), smooth elliptic problems (Poisson, Laplace), and inverse problems (parameter inference).

Recent research explores hybrid approaches combining classical and ML methods, leveraging strengths of both paradigms.

### 5.4 Limitations of This Study

**Single test case:** Conclusions are specific to 1D Burgers' with  $\nu = 0.01$ . Smooth problems ( $\nu = 0.1$ ) or higher dimensions may favor PINNs differently.

**PINN hyperparameters:** While we tested multiple architectures and loss weights, the vast hyperparameter space was not exhaustively searched. Further tuning might improve (but likely not eliminate) the accuracy gap.

**No exact solution comparison:** The Cole-Hopf transformation provides analytical solutions for this problem. Future work should validate against this ground truth rather than high-resolution FD.

## 6 Conclusion

This study presented a comparison of finite difference and physics-informed neural network methods for solving the one-dimensional viscous Burgers' equation. Our findings demonstrate that for this 1D problem exhibiting quasi-shock formation:

1. **Finite difference is decisively superior**, achieving 10-100 $\times$  better accuracy in 500 $\times$  less time
2. **PINN limitations are fundamental**, arising from smooth approximation of non-smooth solutions rather than implementation deficiencies
3. **Method selection should be problem-specific**, with classical methods preferred for low-dimensional shock-capturing and PINNs better suited for high-dimensional or inverse problems

The PINN's persistent PDE residual ( $\sim 0.01$ ) despite excellent IC/BC satisfaction ( $\sim 10^{-5}$ ) provides clear evidence of approximation-theoretic limitations. This result validates the continued necessity of classical numerical methods for important problem classes.

Future research directions include:

- Extension to 2D/3D Burgers' to assess dimensional scaling
- PINN architecture modifications for non-smooth problems (e.g., adaptive activation functions)

This work contributes to the ongoing dialogue between traditional numerical analysis and data-driven scientific computing, emphasizing the importance of critical evaluation over enthusiasm for new methods.

---

## References

- [1] M. Raissi, P. Perdikaris, and G.E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378:686-707, 2019.
- [2] J.M. Burgers, *A mathematical model illustrating the theory of turbulence*, Advances in Applied Mechanics, 1:171-199, 1948.
- [3] R.J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.
- [4] G.E. Karniadakis et al., *Physics-informed machine learning*, Nature Reviews Physics, 3:422-440, 2021.
- [5] H. Bateman, *Some recent researches on the motion of fluids*, Monthly Weather Review, 43(4):163-170, 1915.